МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки і комп’ютерних технологій

Звіт

про виконання лабораторної роботи №11

з курсу «Вибрані розділи науки про дані»

«Векторна база даних та RAG
на матеріалах лабораторної роботи №8»

Виконав:
студент групи ФЕІм-13с
Шевчук Максим Вікторович

Перевірив:
Парубочий В. О.

Львів 2026

Мета та зв’язок із лабораторною роботою №8

Мета роботи — доповнити матеріали енергетичної бази знань із ЛР8 векторним пошуком і RAG: розбити документи на фрагменти, побудувати індекс, сформувати відповіді з джерелами та порівняти якість пошуку.

У ЛР8 вже створено граф Neo4j, виконано імпорт онтології й фактичних даних, підготовлено Cypher-запити та досліджено індексацію [1, розд. 2–12]. Тут ці результати є джерелами корпусу. Хід повторного імпорту та налаштування Neo4j не входить до поточного дослідження.

Таблиця 1 — Що додано до ЛР8

Основа з ЛР8	Нове в ЛР11
Граф та онтологія	Текстовий корпус із метаданими
Пошук за властивостями і зв’язками	BM25, embeddings, FAISS та RRF
Відповідь Cypher	RAG-відповідь із цитатою або відмовою
Час і DB Hits	Recall@1, @3, @5 і перевірка цитат

Виконано п’ять нових результатів: chunking; векторний індекс; RAG; порівняння пошуку; аналіз помилок і перевірка покращення. Експерименти проведено локально на CPU, seed=42. Замість зовнішнього LLM API використано локальну FLAN-T5-small; це відмінність від переліку інструментів у завданні [3].

1 Корпус і дві стратегії chunking

Корпус містить 22 логічні документи: D01–D20 — нові англомовні тематичні технічні нотатки за фактами ЛР8; D21 — основний текст звіту [1]; D22 — правила відображення онтології [2]. Нотатки є похідними власними матеріалами, а не незалежними зовнішніми статтями. Кожна має посилання на джерело; оригінали збережено з SHA-256.

Із PDF прибрано номери сторінок і зайві пробіли. Токени визначає tokenizer MiniLM. Фіксоване вікно містить до 400 токенів, перекриття — 60 токенів (15%). Вікна не перетинають меж PDF-сторінок. Структурний варіант зберігає розділи й абзаци; довгі блоки також обмежені 400 токенами.

Таблиця 2 — Результати розбиття

Стратегія	Фрагменти	Середня довжина
Фіксована	38	207,00 токена
Структурна	72	104,43 токена

Метадані фрагмента: id, doc_id, section, page, language, source та source_page. Для MD page=1 означає умовну сторінку; source_page посилається на вихідний звіт. D21 зберігає справжні номери PDF-сторінок.

Приклад невдалого розрізу: fixed-0034 завершується всередині ідентифікатора «OntologyPro», а fixed-0035 починається з «язок UPPER_SNAKE_CASE». Інший фрагмент закінчується після «numericValue і». Перекриття повертає сусідній контекст, але саме по собі не забезпечує завершені речення. Структурний поділ розділяє факт та його пояснення; це може погіршити їх спільне знаходження.

2 Векторний індекс та фільтрація

Для embeddings використано paraphrase-multilingual-MiniLM-L12-v2: 384 виміри, максимальна довжина входу встановлена 512 токенів. Фрагменти до 400 токенів уміщуються зі спеціальними токенами. Вектори L2-нормалізовано; IndexFlatIP ранжує за внутрішнім добутком, який для таких векторів дорівнює косинусній схожості.

Для обох стратегій збережено vectors.faiss, embeddings.npy та metadata.json. Повторне читання індексу відтворило результати контрольного пошуку. Норми векторів перевірено з похибкою менше 10^{-5}. Фільтри language/doc_id застосовано перед top-k: для дозволених кандидатів створюється точний підіндекс.

Таблиця 3 — П’ять перевірок dense для структури

Питання	Очікується	Документи top-3
Q01	D01	D01, D01, D21
Q02	D02	D02, D21, D02
Q03	D03	D03, D21, D07
Q04	D04	D04, D21, D21
Q05	D05	D05, D05, D21

У цих п’яти запитах цільова нотатка потрапила до top-3. Перевірено language=en, language=uk, document=D15 та порожню вибірку language=de; усі результати відповідають фільтрам. Відсутня мова повертає порожній список.

На відміну від range-індексів ЛР8 [1, розд. 10], цей індекс порівнює текстові подання. Навіть близький за змістом фрагмент може містити інший аспект питання, тому якість оцінюється окремо від працездатності індексу.

3 RAG і автоматична перевірка цитат

Конвеєр: запит → embedding → гібридний пошук → контекст → FLAN-T5-small → перевірка підтвердження → відповідь із [id] або «Недостатньо даних». Фінальна конфігурація використовує fixed, language=en і top-1. Одне джерело скорочує контекст для малої локальної моделі; перевірки top-3 для пошуку виконано окремо.

Промпт вимагає відповідати лише з контексту, копіювати короткий точний фрагмент і відмовляти за відсутності даних. Інструкції всередині джерела трактуються як дані. Еталонна відповідь не передається ні моделі, ні пошуку. Застосовано greedy decoding, max_new_tokens=70.

FLAN-T5 часто повертає тільки число. Тому підтверджений фрагмент розгортається до повного речення того самого джерела. Це екстрактивне постопрацювання, а не додаткова генерація. Ідентифікатор цитати додає програма лише після збігу з реально отриманим фрагментом. Якщо запитаний рік відсутній у контексті, відповідь відхиляється.

Приклад Q01: сирий вихід моделі — «43746». Підтверджена відповідь — «The complete Lab8 graph contains 43746 nodes and 199287 relationships. [fixed-0001]». Числа походять із ЛР8 [1, розд. 3], а новим результатом є їх знаходження та повернення з перевіреним посиланням.

Рядковий збіг доводить наявність цитованого тексту, але не гарантує логічної відповідності будь-якому питанню. Повноту перевірено додатково на заданих восьми відповідних питаннях; механізм відмови — на двох питаннях без відповіді.

3 Результати десяти RAG-запитів

Таблиця 4 — Фінальні відповіді та джерела

ID	Зміст відповіді	Цитата
Q01	43 746 вузлів; 199 287 зв’язків	fixed-0001
Q03	34 085 повних годин	fixed-0003
Q04	9 530 тестових прогнозів	fixed-0004
Q07	502 пропущені години	fixed-0007
Q09	Обмеження lab8_entity_id	fixed-0009
Q11	36 зв’язків SUBCLASS_OF	fixed-0011
Q15	MAE: 0,335966 та 0,423517 kWh	fixed-0015
Q20	forecastKwh > 1,5613	fixed-0020
U01	Номер фізичного лічильника: недостатньо даних	Відмова
U02	Споживання в січні 2025: недостатньо даних	Відмова

У таблиці наведено стислий український переказ; точні англійські відповіді, промпти та сирі виходи є в results/rag.json. Усі 8 прийнятих цитат підтверджено текстом отриманих джерел. Повні відповіді містять потрібні числа або назви. Обидва питання без відповіді завершилися відмовою.

У початковій конфігурації structure/top-3 без розгортання речення повними були 4 із 8 відповідей. Фінальна дала 8 із 8. Одночасно змінено контекст, chunking і постопрацювання, тому цей результат не ізолює внесок кожної зміни. Початкові відповіді збережено в rag_initial.json.

4 Порівняння BM25 dense та RRF

BM25 і dense працюють з однаковими фрагментами кожної стратегії. Гібрид використовує RRF: score(d)=Σ 1/(60+rank(d)), де ранг починається з 1. Для 20 заздалегідь підготовлених питань визначено одну цільову тематичну нотатку. Recall@k тут дорівнює частці питань, для яких її знайдено серед перших k фрагментів.

Таблиця 5 — Фактичний recall@k

Chunking	Метод	@1	@3	@5
fixed	bm25	0,80	1,00	1,00
fixed	dense	0,90	0,95	1,00
fixed	hybrid	1,00	1,00	1,00
structure	bm25	0,90	1,00	1,00
structure	dense	0,95	0,95	0,95
structure	hybrid	0,95	1,00	1,00

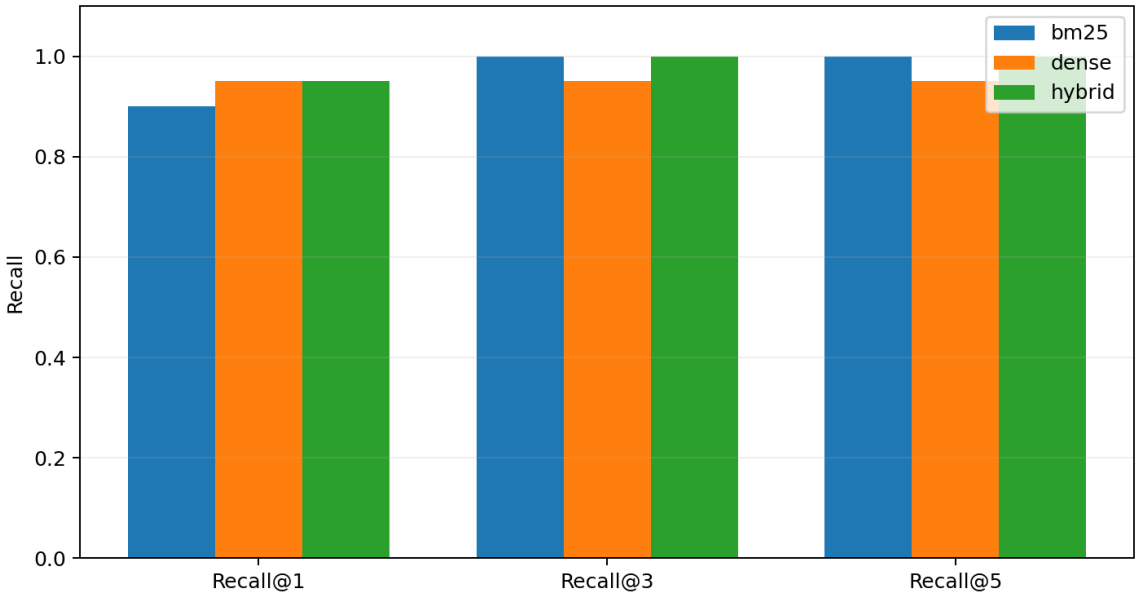


Рис. 1 — Recall для структурного розбиття на 20 питаннях

Рис. 1 показує результати структурного варіанта. Фіксований гібрид має recall@1=1,00; це результат на даному малому наборі, а не гарантія на нових питаннях.

4 Інтерпретація та межі порівняння

На структурному розбитті BM25 помиляється на першій позиції для Q04 та Q08. В обох випадках запит використовує слово population, яке зустрічається і в описі тестових прогнозів, і в описі географічної вибірки. Dense знаходить потрібні нотатки першими; це конкретний приклад виграшу семантичного пошуку на синонімічному формулюванні.

Для Q14 dense віддає перевагу українському першоджерелу D21 та опису одиниць D05. Гібрид повертає цільову D14 другою. Після об’єднання факту з поясненням у fixed D14 стає першою. Отже, надмірне дроблення короткої нотатки іноді погіршує пошук.

Для питань із назвами constraint, SUBCLASS_OF, Q7 та forecastKwh пошукові методи використовують точні технічні терміни. У цьому наборі немає підстав стверджувати загальну перевагу BM25 для всіх кодів або dense для всіх довгих запитів: висновок обмежено реально перевіреними прикладами.

Важлива межа метрики: D21 або D22 іноді містять правильний факт, але не є призначеною тематичною нотаткою D01–D20. Тому така знахідка не зарахована. Оцінюється навігація до конкретного документа, а не всі можливі релевантні відповіді. У results/retrieval.json збережено повні top-5 і тип кожного питання.

Нотатки та питання мають спільне походження з ЛР8. Порівняння конфігурацій проведено на тих самих 20 питаннях, без окремої незалежної тестової вибірки після налаштування. Підсумкові значення слід вважати дослідницькою перевіркою реалізації.

5 Помилки та перевірене покращення

Таблиця 6 — Причини й перевірки

Категорія	Спостереження
Немає в корпусі	U01: серійний номер невідомий; U02: немає даних за 2025 рік. Отримано відмови.
Chunking	Q14: факт та пояснення розділено. Fixed підняв цільову нотатку на 1-ше місце.
Embeddings / ранг	Q14 dense: знайдено близькі першоджерела, але не призначену D14.
Фільтр	language=de закономірно повертає порожній список; це не збій embeddings.
Промпт / модель	Початковий RAG повертав короткі неповні фрагменти; підтримане речення відновлює контекст.

Основне перевірене покращення — використати fixed замість structure у гібридному пошуку. На тих самих 20 питаннях recall@1 зріс із 0,95 до 1,00: +0,05, тобто +5 відсоткових пунктів або одне додаткове питання. Recall@3 та @5 залишилися 1,00. Це окреме порівняння змінює лише стратегію chunking.

Додатковий дослід: залишення найкращого фрагмента кожного документа прибрало повтори джерел. Для structure/dense recall@5 зріс із 0,95 до 1,00; для hybrid виграшу не виявлено. Результати містяться в improvement.json. Фінальний RAG: fixed, RRF, англійські джерела, top-1, локальна FLAN-T5-small та перевірка цитованого речення.

Висновок

До бази знань із ЛР8 додано текстовий пошук і RAG. Побудовано два набори фрагментів та збережені FAISS-індекси, реалізовано фільтрацію, BM25, dense і RRF. На 20 питаннях фіксований гібрид знаходить цільову нотатку першим у 20 випадках; структурний — у 19. Для десяти RAG-запитів отримано вісім підтверджених відповідей і дві відмови.

Нові результати показують вплив chunking і способу подання контексту. Факти про Neo4j, енергоспоживання та похибки прогнозу запозичено з ЛР8 із посиланнями; вони не видаються за нові вимірювання. Обмеження: малий похідний корпус, відсутність незалежного тесту після налаштування, локальна мала мовна модель та обмежена сила рядкової перевірки цитат.

Джерела

[1] Шевчук М. В. Лабораторна робота №8 «Збереження знань у графовій базі даних для візуалізації енергетичних даних». 2026. Вихідний звіт: D:/LNU/DS/8; копія в пакеті: sources/lab8/report.pdf. Посилання в тексті ведуть на розділи цього звіту.

[2] Матеріали ЛР8: MODEL_MAPPING.md, validation.json і benchmark_comparison.json від запуску sandbox_20261016T054342525797Z. Копії: sources/lab8/.

[3] ЛР11_завдання.pdf. «Векторна база даних та RAG», 2026–2027 н. р. Копія: sources/assignment11.pdf.

Декларація використання ШІ

OpenAI Codex використано для підготовки коду, нотаток, питань і звіту; FLAN-T5-small — як компонент RAG. Код фактично запущено, результати та цитати перевірено. Промпти й сирі відповіді збережено. Повна декларація, початковий запит користувача та спосіб перевірки наведені в AI_DECLARATION.md.

Додаток А Розбиття тексту

run_lab11.py — make_chunks; форматування для читання

```
def make_chunks(documents, tokenizer, strategy):
    chunks = []

    def emit(doc, text, section, page):
        encoded = tokenizer(
            text,
            add_special_tokens=False,
            return_offsets_mapping=True,
            truncation=False,
        )
        offsets = encoded["offset_mapping"]
        stride = 340
        for start in range(0, len(offsets), stride):
            end = min(start + 400, len(offsets))
            a = offsets[start][0]
            b = offsets[end - 1][1]
            fragment = text[a:b]
            chunks.append(
                {
                    "id": f"{strategy}-{len(chunks)+1:04d}",
                    "doc_id": doc["id"],
                    "section": section,
                    "page": page,
                    "language": doc["language"],
                    "source": doc["source"],
                    "source_page": doc.get("source_page", page),
                    "title": doc["title"],
                    "text": fragment,
                    "token_count": end - start,
                    "bad_cut": bool(
                        start and a and text[a - 1] not in "\n.!? "
                    ),
                    "start_offset": a,
                    "end_offset": b,
                }
            )
            if end == len(offsets):
```

Додаток А Розбиття тексту — продовження

run_lab11.py — make_chunks; форматування для читання

```
            break

for doc in documents:
    if strategy == "fixed":
        # Page-aware windows: a chunk never gets an invented
single-page locator.
        grouped = {}
        for section in doc["sections"]:
            pg = section.get("page", doc["page"])
            grouped.setdefault(pg, []).append(
                section["heading"] + "\n" + section["text"]
            )
        for pg, parts in grouped.items():
            emit(
                doc,
                "\n\n".join(parts),
                "Whole document page",
                pg,
            )
    else:
        for section in doc["sections"]:
            # Headings/paragraphs define boundaries; long sections
are bounded windows.
            parts = [
                p.strip()
                for p in re.split(r"\n\s*\n", section["text"])
                if p.strip()
            ]
            for part in parts:
                emit(
                    doc,
                    part,
                    section["heading"],
                    section.get("page", doc["page"]),
                )
return chunks
```

Додаток Б Пошук і RRF

run_lab11.py — rank; форматування для читання

```
def rank(
    self,
    query,
    method="hybrid",
    language=None,
    document=None,
    diverse=False,
):
    eligible = [
        i
        for i, c in enumerate(self.chunks)
        if (not language or c["language"] == language)
        and (not document or c["doc_id"] == document)
    ]
    if not eligible:
        return []
    # Filter BEFORE top-k; filtering an already limited list would lose
     valid matches.
    qv = self.encoder.encode(
        [query], normalize_embeddings=True, show_progress_bar=False
    ).astype("float32")
    sub = faiss.IndexFlatIP(self.vectors.shape[1])
    sub.add(self.vectors[eligible])
    ds, di = sub.search(qv, len(eligible))
    dense = [eligible[int(i)] for i in di[0]]
```

Додаток Б Пошук і RRF — продовження

run_lab11.py — rank; форматування для читання

```
    sparse_scores = self.bm25.get_scores(words(query))
    sparse = sorted(eligible, key=lambda i: (-sparse_scores[i], i))
    if method == "dense":
        result = [(i, float(ds[0][n])) for n, i in enumerate(dense)]
    elif method == "bm25":
        result = [(i, float(sparse_scores[i])) for i in sparse]
    else:
        score = {i: 0.0 for i in eligible}
        for ranking in [dense, sparse]:
            for rank, i in enumerate(ranking, 1):
                score[i] += 1 / (60 + rank)
        result = sorted(score.items(), key=lambda x: (-x[1], x[0]))
    if diverse:
        seen = set()
        selected = []
        for i, s in result:
            doc = self.chunks[i]["doc_id"]
            if doc not in seen:
                selected.append((i, s))
                seen.add(doc)
        result = selected
    return [
        {"chunk": self.chunks[i], "score": s} for i, s in result
    ]
```

Додаток В Перевірка та прив’язка цитат

run_lab11.py — citation; форматування для читання

```
support = [
    x["chunk"]
    for x in found
    if norm(raw) and norm(raw) in norm(x["chunk"]["text"])
]
negative = bool(
    re.search(
        r"unknown|not known|insufficient|not provided|unavailable|not
    specified",
        raw,
        re.I,
    )
)
dates = re.findall(r"\b(?:19|20)\d{2}\b", q["question"])
missing_date = any(y not in context for y in dates)
accepted = bool(support) and not negative and not missing_date
quote = raw
if accepted:
    # Expand a short generated span to its complete source sentence.
    This is
    # explicit extractive post-processing, not additional LLM
    generation.
    sentences = re.split(
        r"(?<=[.!?])\s+(?=[A-Z])|\n+", support[0]["text"]
    )
    quote = next(
        (s for s in sentences if norm(raw) in norm(s)), raw
    )
    assert norm(quote) in norm(support[0]["text"])
answer = (
    f"{quote} [{support[0]['id']}]"
    if accepted
    else "Недостатньо даних"
)
```